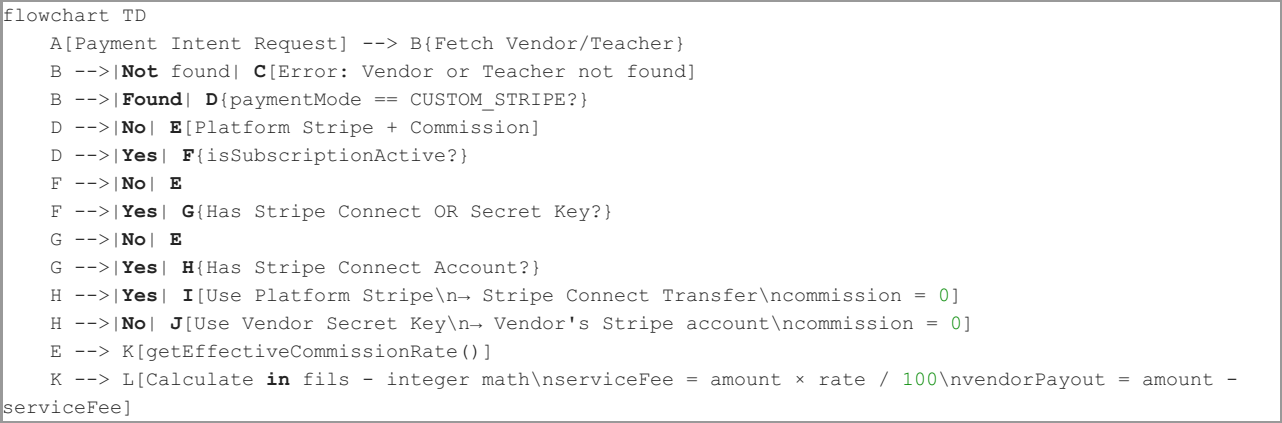


Payment Flow

Gema Event Management Platform
Generated: 2026-02-25

1. Payment Routing Decision

The platform supports two payment modes for vendors/teachers:



2. Create Payment Intent Flow



3. Commission Calculation

```

flowchart LR
    A["amount (e.g. AED 100.00)"] --> B["× 100 → amountInFils (10000)"]
    B --> C["× commissionRate% → serviceFeeInFils (Math.round)"]
    C --> D["÷ 100 → serviceFee (AED)"]
    D --> E["vendorPayout = amount - serviceFee"]
    E --> F["platformCommission = serviceFee"]

```

Integer math in fils (smallest AED unit) prevents floating-point rounding errors.

Scenario	Commission	Notes
Platform Stripe (default)	<code>getEffectiveCommissionRate()</code>	From <code>CommissionConfig</code> model
Custom Stripe (subscribed)0%		Vendor pays subscription instead

4. Stripe Webhook Flow

```

sequenceDiagram
    participant Stripe as Stripe
    participant WH as Webhook Handler
    participant DB as MongoDB
    participant TQ as TicketQueue (BullMQ)
    participant EQ as EmailQueue (BullMQ)

    Stripe->>WH: POST /api/payments/webhook
    WH->>WH: stripe.webhooks.constructEvent()\nverify signature
    WH-->>Stripe: 400 Invalid signature
    WH->>WH: Switch on event.type
    alt payment_intent.succeeded
        WH->>DB: Order.findOne({paymentIntentId})\nupdate paymentStatus='paid'
        WH->>DB: CommissionTransaction.create()
        WH->>TQ: Queue: generateTickets(orderId)
        WH->>EQ: Queue: sendOrderConfirmation(orderId)
        WH-->>Stripe: 200 OK
    else payment_intent.payment_failed
        WH->>DB: Order.update({paymentStatus: 'failed'})
        WH->>EQ: Queue: sendPaymentFailedEmail()
        WH-->>Stripe: 200 OK
    else charge.refunded
        WH->>DB: Order.update({paymentStatus: 'refunded'})
        WH-->>Stripe: 200 OK
    end
end

```

5. Vendor Payout Flow

```

sequenceDiagram
    participant Admin as Admin
    participant AC as AdminPayoutController
    participant PS as PayoutService
    participant SC as StripeConnectService
    participant Stripe as Stripe
    participant DB as MongoDB

    Admin->>AC: POST /api/admin/payouts/process
    AC->>PS: PayoutService.processPayout(vendorId, amount)
    PS->>DB: Vendor.findById(vendorId) - get stripeConnectAccountId
    PS->>DB: Check pending CommissionTransactions
    PS->>Stripe: stripe.transfers.create({amount, destination: connectAccountId})
    Stripe-->>PS: transfer object
    PS->>DB: Payout.create({vendorId, amount, stripeTransferId, status:'completed'})
    PS->>DB: RevenueTransaction.create() - audit log
    PS->>DB: CommissionTransactions.updateMany({status:'paid'})
    PS-->>AC: payout result
    AC-->>Admin: 200 Payout processed

```

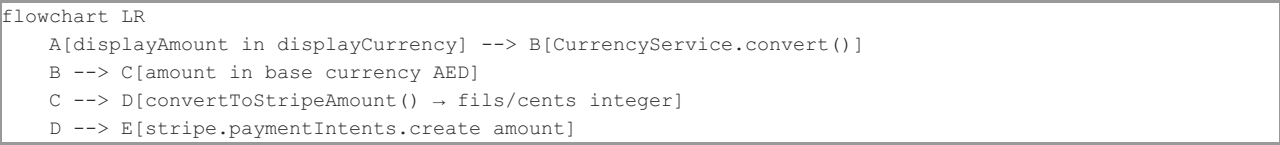
6. Refund Flow



Refund policy:

- **Refundable:** order.subtotal - couponDiscount
- **Non-refundable:** serviceFee + tax
- order.paymentStatus !== 'paid' → refundAmount: 0

7. Multi-Currency Support



CreatePaymentIntentParams includes displayCurrency and displayAmount for UI display while Stripe receives the converted base currency amount.

8. Key Models

Model	Purpose
Order	Core order record — paymentStatus, paymentIntentId, subtotal, serviceFee, tax
Payment	Stripe payment record — paymentIntentId, chargeId, status
CommissionTransaction	Platform commission per order — linked to Order, Vendor, CommissionConfig
RevenueTransaction	Full audit log of all money movement
Payout	Vendor payout records — stripeTransferId, amount, status
CommissionConfig	Per-vendor commission rates (overrides global default)